

Computer Graphics: Richtlijnen voor de finale inzending

Jakob Struye
Tim Apers

In dit document overlopen we nog eens de richtlijnen voor het correct indienen van het practicum voor het vak Computer Graphics, en geven we nog wat bijkomende info over de evaluatie.

Richtlijnen voor een geldige inzending

In deze sectie worden de vereisten besproken waaraan je inzending moet voldoen. Inzendingen die niet aan deze vereisten voldoen kunnen worden beschouwd als niet ingeleverd.

Je inzending moet ten laatste op *25 mei 2025* om *23:59u* via *Blackboard* ingediend worden. Na dit tijdstip verdwijnt de uploadzone op BlackBoard en wordt je inzending niet meer aanvaard. Je kan je Grafische Engine meerdere keren inleveren. Als je na het inleveren van je Engine dus nog fouten ontdekt dan kun je, zolang de uploadzone beschikbaar is, alsnog een verbeterde versie insturen. **Let op:** wij kijken enkel de versie na die het laatst werd ingestuurd!

Je inzending moet bestaan uit één enkel *.tar.gz* (of *.tgz*)-bestand. Andere formaten zoals *.zip* of *.rar* worden *niet* aanvaard. Dit *.tar.gz* bestand moet als naam *sXXXXXXX.tar.gz* (waarbij XXXXXXX de laatste 7 cijfers van je rolnummer zijn) hebben en onder andere aan de volgende vereisten voldoen:

- Het *.tar.gz* bestand moet *alle* sourcebestanden bevatten die je Engine nodig heeft om te werken. Dit betekent dat je niet alleen de code van je Engine zelf maar ook de libraries die voor jullie voorzien werden (bv. *ini_configuration*) meeleverd, ook al heb je deze niet aangepast.
- Voeg geen niet-compilerende code aan je *.tar.gz* bestand toe. Je kan beter een inzending insturen waarvan de code wel compileert maar die niet alle functionaliteit implementeert dan een inzending die niet compileert. C++-code moet compileren met g++13 onder Linux. Als referentie-platform worden de linux-distributie en g++ compiler gebruikt die in de PC-klassen is voorzien.
- Het *.tar.gz* bestand mag geen afbeeldingen of gecompileerde code bevatten. Zorg er dus voor dat er geen *.o*, *.bmp*, *.png*, . . . bestanden of executables in de archive aanwezig zijn. De enige uitzondering hierop zijn textures die gebruikt worden voor texture mapping. Wie CLion gebruikt, verwijdert de *cmake-build-debug* map.
- Op Blackboard hebben we voor de meeste opgaves een aantal voorbeeld *.ini* bestanden beschikbaar gemaakt waarmee je je Engine kon testen. Zorg ervoor dat deze bestanden niet in het *.tar.gz* bestand aanwezig zijn
- Voeg voor de opdrachten waarvoor geen *.ini* bestandsformaat werd opgegeven en voor eventuele extra functionaliteit wél voldoende *.ini* bestanden toe zodat we kunnen zien dat de verschillende aspecten hiervan goed werken; je moet uiteraard niet overdrijven. Je geeft de *.ini* bestanden best ook een naam die beschrijft wat het *.ini* bestand demonstreert, bijvoorbeeld: *stochastisch_lsysteem.ini* of *texture_bol.ini*.

- Voorzie een *readme* (`README.md` of gelijkaardig) waarin je oplijst welke functionaliteit aanwezig is. Bij functionaliteit waarvoor je zelf `.ini`-files voorziet, beschrijf je deze kort.

Je `.tar.gz` bestand moet, bij C++, ook één `CMakeLists.txt`-bestand bevatten waarmee we je Engine eenvoudig kunnen compileren. Dit bestand moet aan de volgende voorwaarden voldoen:

- De `CMakeLists.txt` moet in de root van het `.tar.gz` bestand staan.
- De `CMakeLists.txt` moet werken met CMake 3.28.
- Met opeenvolgend de `cmake .` en `make` commando's moet je engine te compileren zijn. De gegenereerde executable moet **engine** heten en moet in dezelfde map gegenereerd worden als waar het `make` commando wordt uitgevoerd.
- De gegenereerde Makefile moet gebruik maken van de `$(CXX)` omgevingsvariabelen. Hardcode het pad of naam van de compiler dus niet in de `CMakeLists.txt` aangezien deze kan verschillen van platform tot platform.

De `CMakeLists.txt` op BlackBoard voorzien, voldoet aan de technische vereisten, zolang je er niets aan verandert (op het uitbreiden van de lijst met te compileren bestanden na, uiteraard).

Je Grafische Engine moet aangesproken kunnen worden via de interface van `engine.cc/py`. Concreet betekent dit dat de functie `generate_image` geïmplementeerd moet zijn. Hierbij moeten de volgende regels in acht genomen worden:

- Wanneer je engine wordt opgeroepen de namen van één of meer `ini`-bestanden als parameters, moet er per `ini`-bestand een `.bmp` bestand worden gegenereerd met, op de extensie na, dezelfde naam.
- Dit `.bmp`-bestand komt terecht in de map waaruit de engine werd opgeroepen. Als er extra files nodig zijn (bv. een `L2D`-bestand), dan worden die ook enkel in de huidige map gezocht.
- Wanneer een bepaalde image niet gegenereerd kan worden omdat de benodigde functionaliteit niet geïmplementeerd is, moet er geen afbeelding gegenereerd worden. Je kan dit doen door `generate_image` een lege afbeelding te laten returnen. Dit doe je d.m.v. `return img::EasyImage();`.

Overzicht evaluatie

Ter info geven we jullie nog kort mee hoe we jullie engine evalueren.

- We gebruiken een set van enkele honderden afbeeldingen om de correcte werking van alle functionaliteit te evalueren. Deze komen niet noodzakelijk uit de beschikbare `ini`'s op BlackBoard.
- We draaien voor elke afbeelding een aparte instantie van je engine. Een crash in een bepaalde afbeelding heeft dus geen effect op de verdere afbeeldingen.
- Er kunnen meerdere instanties van je engine parallel draaien, en elke instantie die zich niet aan de restricties qua runtime en geheugengebruik (30 minuten, 2GB) houdt, wordt direct gestopt.
- Je mag aannemen dat voor elk onderdeel uit de opgaves enkel punten toegekend worden wanneer deze volledig werken. Voor zaken als een `z`-buffering die meestal de juiste lijn kiest, 3D `L`-systemen die werken zolang er geen `'\'` in de replacement rules staat, of een Mengerspons waarin enkele kubussen ontbreken, worden geen punten toegekend. Wanneer verschillende stukken van één onderdeel apart kunnen voorkomen (bv. de verschillende platonische lichamen) worden er wel voor elk werkend stuk apart punten toegekend.

Veelvoorkomende fouten

Tijdens het jaar zijn we meerdere inzendingen tegengekomen slechts gedeeltelijk of zelfs helemaal niet werkten door kleine foutjes die eenvoudig vermeden hadden kunnen worden. Gezien wij je Engine enkel kunnen evalueren op basis van wat er werkt en niet op basis van 'wat er zou moeten werken' raden we iedereen ten sterkste aan om te controleren of je inzending in orde is vooraleer je de opdracht indient op Blackboard. Om je hierbij te helpen worden hieronder een aantal belangrijke 'valkuilen' opgesomd:

- Zorg ervoor dat je Engine niet 'overijverig' is. Je Engine moet, bij de verplichte opdrachten, enkel doen wat er volgens de opgave verwacht wordt en niets meer. Als het type van een .ini bestand gelijk is aan 'ZBuffering' dan mogen belichting of texture mapping bijvoorbeeld niet worden toegepast. Kijk ook zeker na dat er door latere functionaliteit geen regressies voorgevallen zijn in vroegere opdrachten, zoals volledig zwarte afbeeldingen pre-belichting. Voor alle ini's die doorheen het project voorzien waren, moet je output er nog steeds uitzien zoals de bijgeleverde referentie.
- Controleer dat je .tar.gz bestand in orde is nadat je deze aanmaakt. Dit betekent dat je .tar.gz alle bestanden moet bevatten die nodig zijn om de code te compileren en dat de combinatie van CMake en de daarmee gegenereerde Makefile ook effectief op basis van deze bestanden een werkende executable kan genereren. Kopieer, om te controleren dat dit het geval is, het .tar.gz bestand naar een lege directory, pak hem uit, compileer de code en run je Engine.
- Bij wie voor C++ een andere compiler dan het referentieplatform (g++) gebruikt, kijkt best de compilatie op het referentieplatform na. Soms zijn er bij g++ enkele extra `include` commando's nodig. Wie niet meteen toegang heeft tot een systeem met deze compiler, maakt het best gebruik van de proefcompilatie om dit te controleren. Dit lijkt vooral voor te komen bij Engines die enkel op Mac getest zijn.